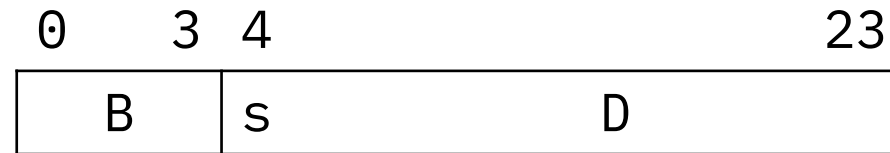


Base-displacement limitations

- Base-displacement, as implemented, has two limitations:
 - It can only address 4096 bytes (displacements 0 to 4095)
 - It cannot reach addresses before the base
- Two other forms of addressing data and instructions remove these limitations and make programming easier:
 - **Long displacement**
 - **Relative addressing**

Long Displacement

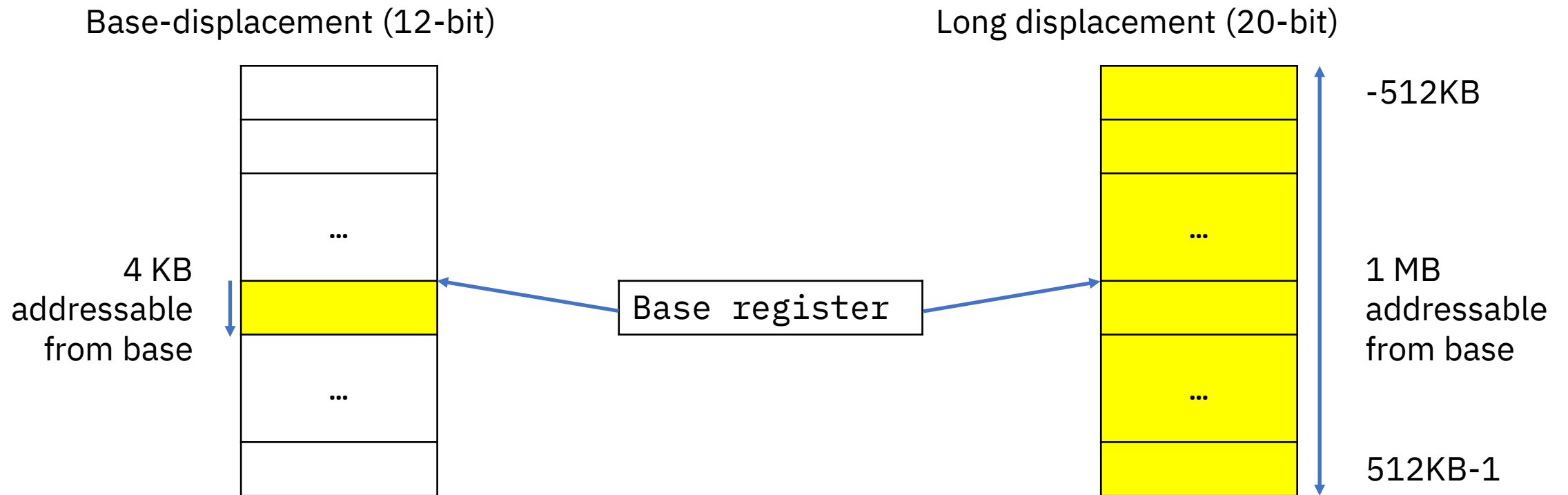
- Long Displacement provides a *20-bit signed displacement*
- Conceptually (this is not the actual format), a long displacement operand address looks like this (**s** is the sign bit):



- It extends the addressable range from 4KB to 1MB (1048576 bytes)
 - From -524288 (hexadecimal **80000**), 512KB *before* the base location
 - To 524287 (**FFFFF**), 512KB-1 beyond the base location

Long Displacement

- Comparing “traditional” base-displacement with long displacement:



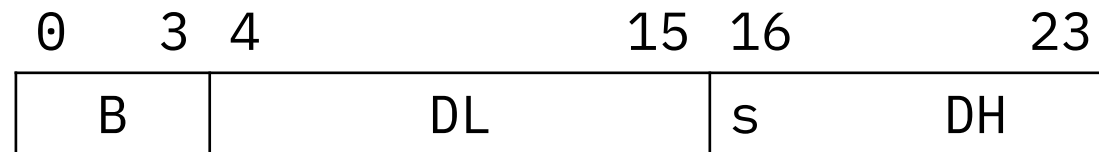
- To address 1MB using 12-bit displacements would require 256 registers!

Long Displacement instructions

- How do we use long displacement?
- Many instructions have a form that uses 12-bit displacement and another that uses 20-bit
 - For example, there is an ADD instruction that uses 12-bit displacement, and another that uses 20-bit
- To use long displacement, the programmer simply chooses the 20-bit version of the instruction
- We'll see examples later

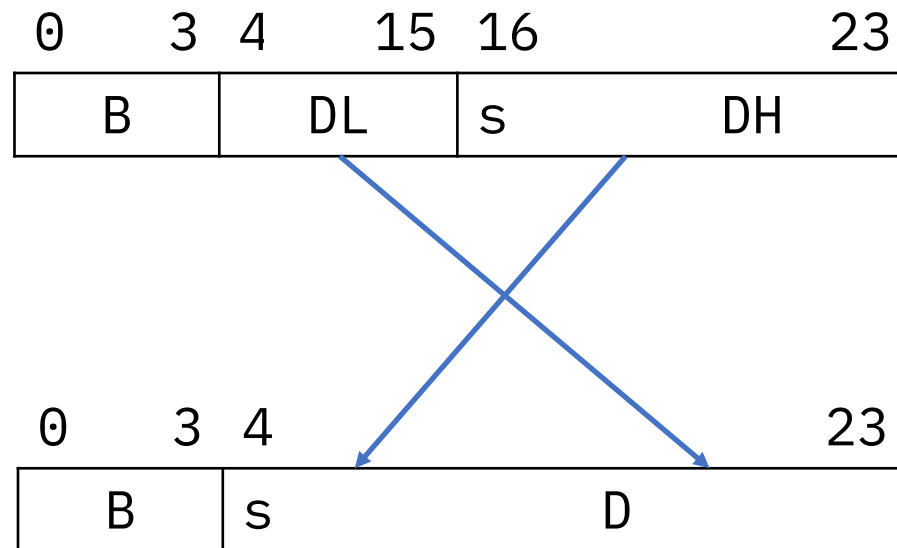
Long Displacement format

- From a programmer's point of view, the mechanism just works. You don't need to know how long displacements are encoded
- However, for completeness, here is the actual long-displacement format:
 - An 8-bit signed prefix is *appended* to the “traditional” addressing halfword
 - The traditional displacement is now called “displacement low” (**DL**)
 - The new signed prefix is called “displacement high” (**DH**)



Long Displacement format

- To compute the effective address:
 1. The displacement fields are rearranged (internally) to form a “proper” base-displacement address



2. The signed displacement is added to the base address

Long Displacement examples

- Example 1. The displacement is 100,000 bytes from the base

- Register 7, the base, contains 00000000 0002A000
- The displacement is **186A0** (decimal 100,000)

- In an instruction, the operand looks like this:

76A018

- The effective address is:

Base	00000000	0002A000
Displacement		+186A0
Effective address	00000000	000426A0

Long Displacement examples

- Example 2. The displacement is 100,000 bytes *before* the base address
 - Register 7, the base, contains 00000000 0002A000
 - The displacement is **E7960** (decimal **-100,000**)

- In an instruction, the operand looks like this:

7960E7

- The effective address is:

Base	00000000	0002A000
Displacement	FFFFFFFF	FFFE7960
Effective address	00000000	00011960

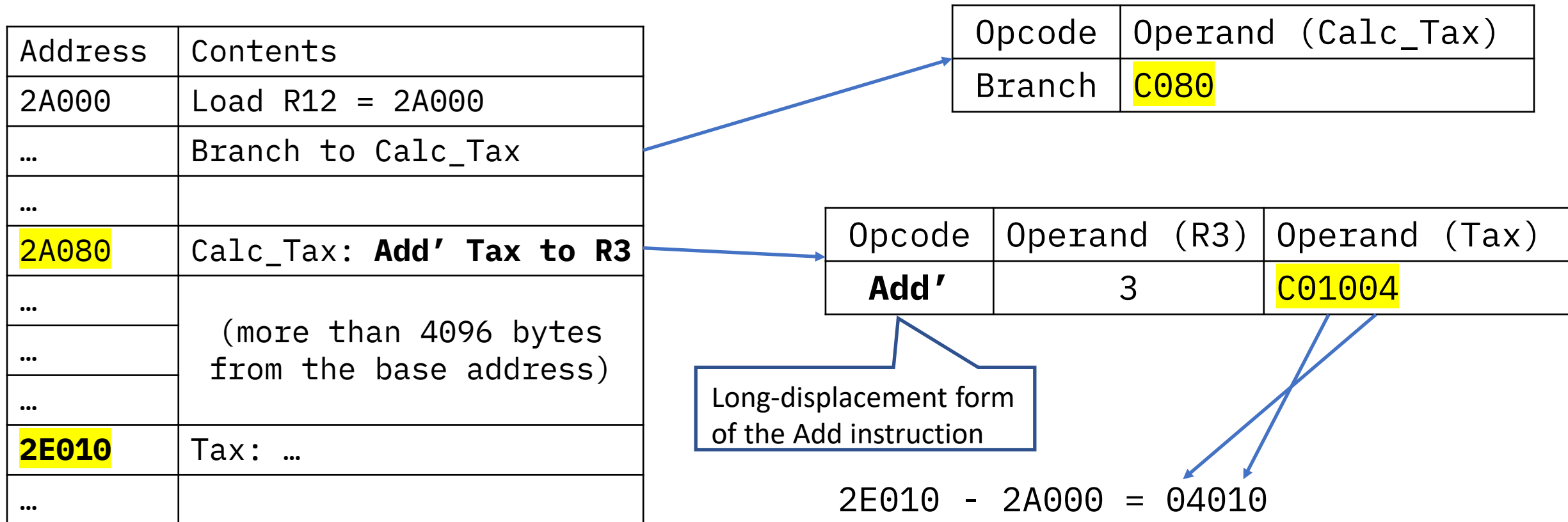
Why negative displacements?

- It's a fair question. Wouldn't it be simpler if we had unsigned (that is, only positive) long displacements?
- One reason for having negative displacements is that some operating system data structures (called, in z/OS, "control blocks") have a *prefix*: an area that precedes the start of the block.

A negative displacement allows the use of a single base register for the main control block ("forward" from the base) and its prefix (before the base)

Long displacement instructions

- Example 3. Our example program using the long-displacement version of “Add”:



Long displacement - summary

- Long displacement brings a 1 MB addressing range from the base address
 - This is 256 times that of “traditional” base-displacement
- Long displacements are signed, so the range covers 512 KB before the base and 512 KB from the base
- Next, we will see the third form of addressing operands, called **relative addressing**